

Oliver Krauss

Networking & REST APIs

ALD | MTD.ba VZ SS26

Hagenberg 16.06.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: DAO & 3-Layer Architecture

Last session: keeping data where it belongs

Clean layered architecture separates UI, business rules, and data access so each can change independently.

Recap: DAO & 3-Layer Architecture

Last session: keeping data where it belongs

Clean layered architecture separates UI, business rules, and data access so each can change independently.

Architecture:

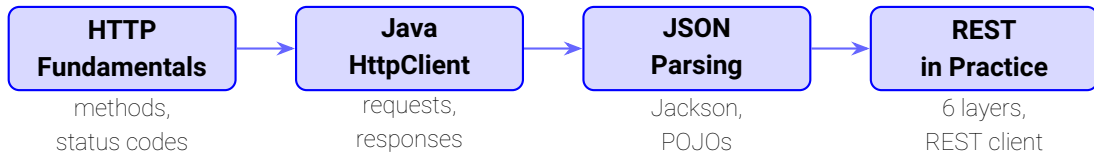
- Presentation / Business Logic / Data Access layers
- Each layer calls only the one directly below
- Benefits: testability, replaceability, readability

DAO Pattern:

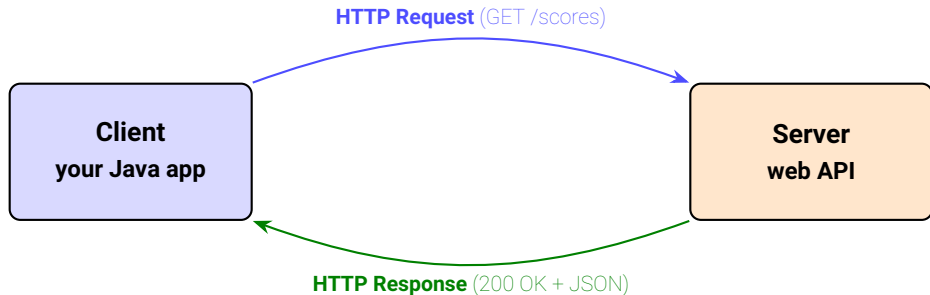
- `Dao<T, K>` interface: `findById`, `findAll`, `save`, `delete`
- JDBC: `Connection`, `PreparedStatement`, `ResultSet`
- Game persistence: `Player`, `Score`, `Item` in MariaDB

Today: Your database stores data *locally*. What if the data lives on a server on the other side of the internet?

Today's Journey



The Web as Client and Server



The client **always initiates**. The server waits, handles the request, sends a response, and forgets. Each exchange is **stateless** -- the server keeps no memory of previous calls.

What travels over the wire?

Plain text (HTTP/1.1) or binary frames (HTTP/2):

Request: method + URL + headers + optional body

Response: status code + headers + optional body

Anatomy of an HTTP Request

A real HTTP request (plain text)

```
1  POST /api/scores HTTP/1.1
2  Host: game.example.com
3  Content-Type: application/json
4  Authorization: Bearer abc123
5
6  {
7    "player": "Ada",
8    "points": 4200
9  }
```

Part	Meaning
POST	HTTP method
/api/scores	resource path
HTTP/1.1	protocol version
Host:	which server
Content-Type:	body format
Authorization:	who is asking
blank line	separator
{...}	request body
<i>Content-Type: application/json</i> tells the server: "the body I am sending is JSON." Without it, the server may not know how to read the body.	

Anatomy of a URL

`https://pokeapi.co/api/v2/pokemon/pikachu`

scheme
(protocol)

host
(server)

path
(resource)

segment
(item)

Query parameters (optional)

Appended after **?** for filtering/sorting:

/scores?player=Ada&limit=10

Each param is **key=value**, separated by **&**.

In Java: `URI.create()`

Type-safe URLs before sending

HttpRequest requires a **URI** object, not a plain **String**. **URI** is a type-safe wrapper that validates the URL format before your request is sent -- a bad URL fails immediately rather than at runtime.

```
URI.create(baseUrl + "/scores/" + id)
```

Concatenate path segments -- Java does not have URL template literals.

HTTP Methods

Method	Action	Typical URL	Body?
GET	Read a resource	/scores/42	No
POST	Create a new resource	/scores	Yes
PUT	Replace a resource	/scores/42	Yes
PATCH	Partially update	/scores/42	Yes
DELETE	Remove a resource	/scores/42	No
HEAD	Like GET, headers only	/scores/42	No

HTTP Methods

Method	Action	Typical URL	Body?
GET	Read a resource	/scores/42	No
POST	Create a new resource	/scores	Yes
PUT	Replace a resource	/scores/42	Yes
PATCH	Partially update	/scores/42	Yes
DELETE	Remove a resource	/scores/42	No
HEAD	Like GET, headers only	/scores/42	No

REST convention

Use nouns in URLs, not verbs:

✓ GET /scores

✗ GET /getScores

The method already is the verb.

Common confusion: PUT vs PATCH

PUT replaces the *entire* resource -- omit a field and it is gone.

PATCH updates only the fields you send.

Example: to update only a player's username, use PATCH -- PUT would also reset their score.

Safe vs. Idempotent

Two properties that describe what happens when you *repeat* the same request.

Safe -- read-only

A safe method **never changes** server state -- it only reads.

Call it as often as you like; nothing on the server is modified.

✓ **Safe:** GET, HEAD

✗ **Not safe:** POST, PUT, PATCH, DELETE

Idempotent -- same end state

Repeating the request leaves the server in the **same state**.

The response may differ (e.g. 404 after DELETE), but the effect does not stack.

✓ **Idempotent:** GET, PUT, DELETE, HEAD

✗ **Not idempotent:** POST, PATCH

Safe vs. Idempotent

Method	Safe?	Idempotent?
GET, HEAD	yes	yes
POST	no	no
PUT, DELETE	no	yes
PATCH	no	no

Why it matters

Safe $\Rightarrow$ browsers and caches may prefetch without side effects.

Idempotent $\Rightarrow$ safe to **retry** after a timeout -- e.g. DELETE twice still leaves one deleted record, but **POST** `/scores` twice creates *two* scores.

HTTP Status Codes

Range	Category	Common codes	Meaning
2xx	Success	200 OK	Everything worked
		201 Created	Resource was created
		204 No Content	OK, no body returned
3xx	Redirect	301 Moved Permanently	URL has changed
4xx	Client error	400 Bad Request	Malformed request
		401 Unauthorized	Missing / bad credentials
		404 Not Found	Resource does not exist
		422 Unprocessable	Validation failed
5xx	Server error	500 Internal Server Error	Server crashed
		503 Service Unavailable	Server overloaded / down

Rule of thumb: **4xx** = your fault; **5xx** = their fault.

REST: Four Core Constraints

1. Resource-based URLs

Every piece of data is a **resource** with a stable address:

`/players/7` `/games/42/scores` --
nouns, not verbs.

Example: `GET /scores/42` retrieves score 42.

2. Stateless

Each request carries *all* the context the server needs. No session stored server-side.

Example: every request includes the auth token -- the server forgets you between calls.

3. Uniform interface

Same methods (GET / POST / PUT / DELETE) work across all resources -- no custom verbs.

Example: `DELETE /players/7` and `DELETE /scores/42` use the same method.

4. Representation

The server sends a **representation** (usually JSON), not the resource itself. The client interprets it.

Example: `{"id":42,"points":4200}` -- a snapshot, not a live Java object.

Uniform Interface: Players & Scores

The **HTTP method** is the verb; the **URL** is the noun.

What you want	Method	URL
List all players	GET	/players
Get one player	GET	/players/7
Register new player	POST	/players
Change player name	PATCH	/players/7
List scores for a player	GET	/players/7/scores
Add a score for a player	POST	/players/7/scores
Get one score	GET	/scores/42
Delete a score	DELETE	/scores/42

Uniform interface

GET, **POST**, **PATCH**, and **DELETE** mean the same thing everywhere.

Only the resource path changes: **/players**, **/players/7/scores**, **/scores/42**, ...

“REST-like” APIs

REST is an **architectural style**, not a certifiable standard -- there is no compliance checklist.

Most production APIs are **REST-like**: they borrow HTTP, JSON, and resource URLs, but bend one or more of the four constraints.

Common bends

- Verbs in paths: **/getScores**, **/search**
- Actions as sub-paths: **POST**
/orders/42/cancel
- POST used for reads (legacy APIs)
- Server-side sessions / operations

What still carries over

- Resources + HTTP methods as the mental model
- JSON representations, not live objects
- Status codes for success vs. failure

Practical takeaway

Design your own APIs RESTfully. When consuming someone else's, **read their docs** -- assume REST-like until proven otherwise.

Interactive Break #1 -- HTTP in Action

For each scenario, pick the **HTTP method** and the expected **success status code**:

1. Retrieve the top-10 leaderboard entries
2. Submit a new score after finishing a level
3. Delete a saved game by its ID
4. Change a player's username (full replacement)
5. Check if a username is already taken (no body needed)

Method	Status
GET	200
POST	201
PUT	200
DELETE	204
HEAD	200

(Methods and common success codes for reference -- not all match!)

Interactive Break #1 -- Answers

#	Scenario	Method	Status
1	Retrieve top-10 leaderboard	GET	200 OK
2	Submit a new score	POST	201 Created
3	Delete a saved game by ID	DELETE	204 No Content
4	Replace player username	PUT	200 OK
5	Check if username is taken	HEAD	200 OK (exists) / 404 (free)

Scenario 5 -- HEAD

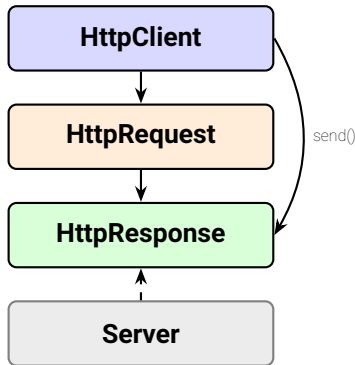
HEAD is like **GET** but returns **only headers, no body**. Useful for existence checks, reducing bandwidth. Many APIs do not support it -- **GET** with a light response is a common fallback.

Java's Built-in HTTP Client

Since Java 11 -- no extra dependency

- `java.net.http.HttpClient` -- the connection manager
- `java.net.http.HttpRequest` -- what you want to send
- `java.net.http.HttpResponse` -- what came back

All three live in the standard library. No Maven dependency needed.



The Builder Pattern

Builder pattern everywhere

Every object is constructed with a fluent builder:

```
HttpClient.newBuilder().....build()
```

```
HttpRequest.newBuilder().....build()
```

You build the object by chaining method calls.

Building an HttpClient and HttpRequest

Create the client

```
1  HttpClient client = HttpClient.  
    newBuilder()  
2    .version(HttpClient.Version.HTTP_2)  
3    .connectTimeout(Duration.ofSeconds  
    (10))  
4    .build();
```

Build a GET request

```
1  HttpRequest request = HttpRequest.  
    newBuilder()  
2    .uri(URI.create(  
3      "https://pokeapi.co/api/v2/  
    pokemon/pikachu"))  
4    .header("Accept", "application/json  
    ")  
5    .GET().build();
```

Build a POST request

```
1  String json = ""  
2    {"player": "Ada", "points": 4200}  
3    "";  
4  
5  HttpRequest post = HttpRequest.  
    newBuilder()  
6    .uri(URI.create(  
7      "https://api.example.com/  
    scores"))  
8    .header("Content-Type",  
9      "application/json")  
10   .POST(HttpRequest.BodyPublishers  
11     .ofString(json))  
12   .build();
```

Our first request (Synchronous)

```
1  HttpResponse<String> response =
2      client.send(request, HttpResponse.BodyHandlers.ofString());
3  int status = response.statusCode();    // e.g. 200
4  String body = response.body();        // JSON string
5  if (status == 200) {
6      System.out.println("Got: " + body);
7  } else {
8      System.err.println("Error " + status);
9  }
```

BodyHandlers

- **ofString()** -- response as **String**
- **ofFile()** -- save to a file
- **ofInputStream()** -- stream the body
- **ofByteArray()** -- raw bytes

Exceptions to handle

client.send() declares:

IOException -- network failure

InterruptedException -- thread interrupted

Full Example: PokéAPI

```
1  public class PokeClient {
2      public static void main(String[] args) throws IOException, InterruptedException
3      {
4          HttpClient client = HttpClient.newBuilder()
5              .connectTimeout(Duration.ofSeconds(10))
6              .build();
7          HttpRequest request = HttpRequest.newBuilder()
8              .uri(URI.create("https://pokeapi.co/api/v2/pokemon/pikachu"))
9              .header("Accept", "application/json")
10             .build();
11         HttpResponse<String> response =
12             client.send(request, HttpResponse.BodyHandlers.ofString());
13
14         System.out.println("Status : " + response.statusCode());
15         System.out.println("Body    : " + response.body().substring(0, 120) + "...");
16     }
```

What is JSON?

A JSON object

```
1  {
2    "name":    "pikachu",
3    "id":      25,
4    "weight":  60,
5    "types": [
6      { "slot": 1, "type": { "name": "
          electric" } }
7    ],
8    "base_experience": 112,
9    "is_default": true,
10   "sprite": null
11 }
```

JSON type	Example
Object	{ "key": value }
Array	[1, 2, 3]
String	"hello"
Number	42, 3.14
Boolean	true, false
Null	null

Why JSON?

Human-readable, language-agnostic, natively understood by JavaScript. The de-facto standard for REST APIs. Think of a JSON object as a Java **Map<String, Object>** written as plain text.

Jackson: the Standard Java JSON Library

Maven dependency

```
1  <dependency>
2    <groupId>com.fasterxml.jackson.core</groupId>
3    <artifactId>jackson-databind</artifactId>
4    <version>2.17.1</version>
5  </dependency>
```

Core class: ObjectMapper

```
1  ObjectMapper mapper = new ObjectMapper();
2
3  // JSON String -> Java object
4  Pokemon p = mapper.readValue(jsonString, Pokemon.class);
5
6  // Java object -> JSON String
7  String out = mapper.writeValueAsString(p);
```

Jackson: Tips & Alternatives

Create once, reuse often

Creating an **ObjectMapper** is slow -- create one instance and reuse it everywhere in your class. Calling **readValue()** on it is fast.

Other popular options

- **Gson** (Google) -- simpler API
- **JSON-B** (Jakarta) -- Java EE standard
- **Moshi** -- Kotlin-friendly

Jackson is the most feature-complete and the default in Spring Boot.

Mapping JSON to a POJO

```
1  public class Pokemon {
2      public String name;    // same name -> no annotation needed
3      public int id;         // same name -> no annotation needed
4
5      @JsonProperty("base_experience") // JSON key differs from Java field
6      public int baseExperience;
7
8      // Jackson requires a no-arg constructor
9      public Pokemon() {}
10 }
```

@JsonProperty

Only needed when the Java field name **differs** from the JSON key.
If they match (e.g. `name` / `"name"`), omit the annotation.

Deserialising the Response

```
1  ObjectMapper mapper = new ObjectMapper();
2  mapper.configure(
3      DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES,
4      false);      // ignore extra fields
5
6  Pokemon pika = mapper.readValue(response.body(), Pokemon.class);
7
8  System.out.println(pika.name); // -> pikachu
9  System.out.println(pika.baseExperience); // -> 112
```

FAIL_ON_UNKNOWN_PROPERTIES = false

PokéAPI returns 50+ fields per Pokémon. Without this setting Jackson throws an exception for every JSON field that has no matching field in your class. Set it once and you can ignore fields you do not need.

Serialising Java to JSON

```
1  ObjectMapper mapper = new ObjectMapper();
2  Score s = new Score("Ada", 4200);
3  String json = mapper.writeValueAsString(s);
4  // -> {"player":"Ada","points":4200}
5
6  HttpRequest post = HttpRequest.newBuilder()
7      .uri(URI.create(baseUrl + "/scores"))
8      .header("Content-Type", "application/json")
9      .POST(HttpRequest.BodyPublishers.ofString(json))
10     .build();
```

For **POST** and **PUT**, serialisation runs in the opposite direction to **GET**: your Java object becomes a JSON **String** via **writeValueAsString**, then that string is the request body. **Content-Type: application/json** tells the server how to parse it -- the same header you already saw on incoming responses.

Error Handling Strategies

Network failure (IOException)

- **Retry with backoff** -- wait a bit, then try again (e.g. 1s, 2s, 4s -- doubling each time)
- **Fallback** -- serve cached data or show offline message
- **Fail fast** -- log and rethrow to the UI

Client errors (4xx)

- **400 Bad Request** -- fix the request
- **401/403** -- fix authentication
- **404** -- you asked for a non-existent resource

Server errors (5xx)

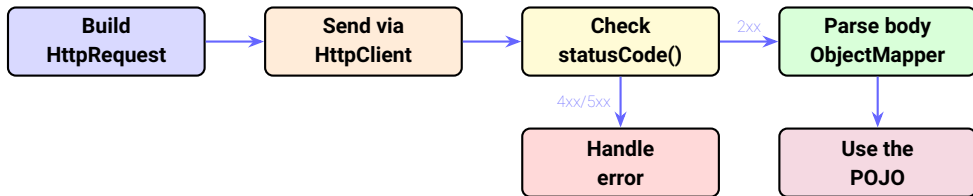
- **500** -- server bug, no point retrying immediately
- **503** -- overloaded, retry with backoff
- Always log the body -- it may contain a useful error message

Timeouts

Always set `connectTimeout` and `HttpRequest.timeout()`. Without them, a hung server blocks your thread forever.

```
.timeout(Duration.ofSeconds(5))
```

The Full Pipeline



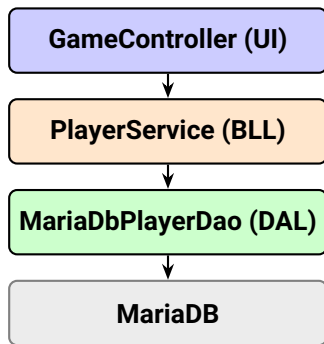
Always check the status code first

Never try to parse the body without checking the status. A 404 body is an error message, not your data. A 500 body may be an HTML error page.

IOException is a network error

The request never reached the server (timeout, DNS failure, no network). This is different from a 5xx -- that means the server responded but failed.

Recap: 3 layers



Each layer calls only the one directly below. Session 10 wired this stack in one JVM.

Today

The data may live on **another machine**. We need **two** three-layer stacks and a wire between them.

Where REST Belongs: Presentation (Server)

REST is **not** mainly “replace JDBC in your DAO.” On a **backend**, REST lives in the **presentation layer**.

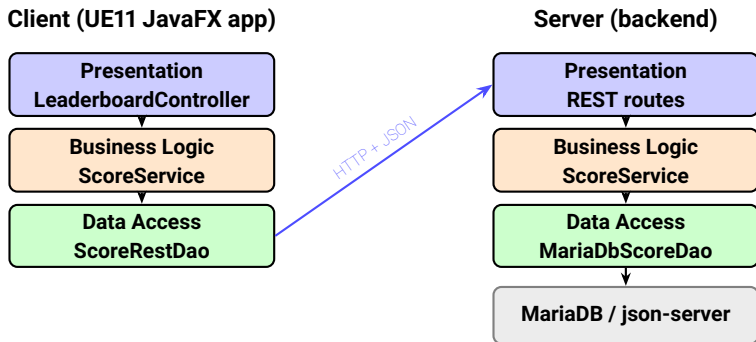
Layer	Local app (UE10)	Backend with REST
Presentation	JavaFX controller	REST API (GET / scores , JSON)
Business Logic	ScoreService	Same rules, unchanged
Data Access	MariaDbScoreDao	Still JDBC -- SQL stays here

The same Business Logic can serve a JavaFX GUI **and** a REST endpoint -- no duplication of rules.

*On the **client**, `ScoreRestDao` sits in the DAL -- but it talks to the server's **presentation**, not to the database.*

Two Applications, Six Layers

3 layers on the client + 3 layers on the server = **6 layers** separated by the network.



Key idea

The frontend's DAL is a REST **client**. It calls the backend's **presentation layer** -- not the backend's database.

One Request Journey: GET /scores

Client

1. User clicks *Refresh* (presentation).
2. Controller calls `scoreService.getLeaderboard()` (BLL).
3. Service calls `scoreRestDao.findAll()` (DAL).
4. `ScoreRestDao`: build `HttpRequest`, `send()`, check status.
9. Client DAL: `mapper.readValue(...)` deserialises the body.
10. BLL returns `List<Score>`; `TableView` updates.

Server

5. `GET /scores` crosses the network.
6. REST handler receives the request (presentation).
7. Handler $\rightarrow$ service $\rightarrow$ DAO $\rightarrow$ DB.
8. Server returns `200` + JSON array.

Client DAL: ScoreRestDao

```
1  public class ScoreRestDao implements ScoreDao {
2      private final HttpClient client;
3      private final ObjectMapper mapper;
4      private final String baseUrl;
5
6      @Override
7      public List<Score> findAll() throws IOException, InterruptedException {
8          var req = HttpRequest.newBuilder()
9              .uri(URI.create(baseUrl + "/scores"))
10             .build();
11          var resp = client.send(req, HttpResponse.BodyHandlers.ofString());
12          // check status, then mapper.readValue(...)
13      }
14  }
```

Implements **ScoreDao** on the **client** only -- a remote adapter. Business Logic still depends on the interface from Session 10; only the wiring and implementation change.

Interactive Break #2 -- When Do I Need REST?

For each scenario: do you need **REST/HTTP** (remote API), or is **local persistence / in-process** enough?

1. **dungeon game** saves scores to **MariaDB on the same laptop** -- only your JavaFX app reads them.
2. **leaderboard**: class-wide live top-10 from Docker (**json-server**).
3. JavaFX today; tomorrow a **web dashboard** needs the same scores (no duplicated rules).
4. **Startup**: load item definitions from a **CSV file** shipped inside the JAR.
5. **Two packages in one JVM**: **GameController** and **LeaderboardController** both use the **same ScoreService** instance.

Think: Who is the client? Where does the data live? Must it cross a process or network boundary?

Interactive Break #2 -- Answers

#	Scenario	REST?	Why
1	Local MariaDB, single desktop app	No	One JVM; DAL → JDBC is enough.
2	Shared class leaderboard	Yes	Separate processes; HTTP + JSON across the wire.
3	JavaFX + future web UI, same rules	Yes	Server presentation as API; both UIs call it.
4	CSV in JAR	No	Classpath resource, not a remote service.
5	Two UIs, one JVM, one ScoreService	No	Call the service directly -- HTTP is overhead.

Decision rule

REST when the consumer and the data live in **different applications or machines** and you need a **stable, language-neutral contract** (URLs + JSON).

Skip REST when everything runs **in one process** or you only need **local files / JDBC**.

What We Covered Today

1. HTTP & REST

- Client-server, stateless; URLs = nouns, methods = verbs
- Status codes; safe vs. idempotent
- REST constraints; uniform interface; APIs often REST-like

2. Java HttpClient

- **HttpClient** + builder; **client.send()** (sync)
- Check **statusCode()** first; timeouts
- **IOException** $\neq$ 4xx/5xx

3. JSON (Jackson)

- **ObjectMapper.readValue()** / **writeValueAsString()**
- POJOs, **@JsonProperty**, ignore unknown fields

4. REST in Practice

- Send $\rightarrow$ check status $\rightarrow$ parse JSON
- Six layers; **ScoreRestDao** = remote DAL
- Server REST = presentation; 4xx vs. 5xx

Bonus: What This Section Is (and Is Not)

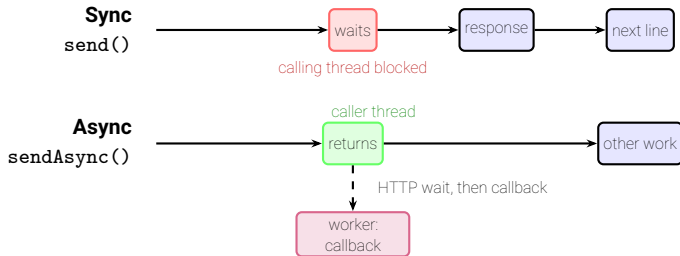
Multithreading (full treatment)

Threads, the JavaFX Application Thread, background workers, **Task<T>**, **ExecutorService**, and production-style **sendAsync** with safe UI updates.

This bonus section

Only enough **thread vocabulary** to see why sync HTTP can freeze a UI and what async changes -- **not** a substitute for Session 13.

Blocking vs. Non-Blocking (Sync vs Async)

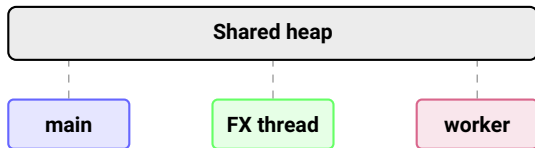


	Synchronous <code>send()</code>	Asynchronous <code>sendAsync()</code>
Calling thread	Stops until response arrives	Continues right away
Result available	Next line after <code>send()</code> returns	CompletableFuture / callback
JavaFX risk	FX thread blocked $\Rightarrow$ frozen UI	FX thread can stay responsive

Sync = wait on the spot. Async = start now, handle the result later.

Threads: Minimal Preview

- A **thread** runs instructions one after another -- while a call **blocks**, that thread cannot do anything else.
- `client.send()` runs on **whatever thread called it** (`main` in a CLI; in JavaFX, often the **FX Application Thread**).
- `client.sendAsync()` hands waiting to HttpClient **worker threads** and returns a `CompletableFuture` immediately.
- Updating `TableView` or labels from a callback may run on a **worker thread**.



sendAsync in Code

Non-blocking send with `CompletableFuture`

```
1  client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
2      .thenApply(HttpResponse::body)
3      .thenAccept(body -> System.out.println("Received: " + body))
4      .exceptionally(e -> { e.printStackTrace(); return null; });
5
6  // calling thread continues here immediately
7  System.out.println("Request sent, waiting for response...");
```

Callbacks may run on a worker thread. For JavaFX, Session 13 shows `thenAcceptAsync(..., Platform::runLater)` or `Task<T>`.

Bonus: When Sync vs. Async?

When async makes sense

- Production -- keep UI responsive
- Many requests in parallel
- Long-running downloads

When sync is the right choice

- learning exercises
- CLI tools; next line needs the result
- Easier to read, debug, and test